

SENAI — TURMA DESENVOLVIMENTO DE SISTEMAS

Projeto Interdisciplinar "Mural"

Rede Social da Escola — construída via GitHub + Supabase, matéria por matéria

Especificação completa das 6 peças da trilha (conceito → comando, por etapa)

Gerado a partir das especificações em [Atividades/Desenvolvimento de Sistemas/](#)

Sumário

1. Visão Geral do Projeto Interdisciplinar
2. Peça 1 — Kickoff do Projeto construída *(Introdução de Desenvolvimento de Projetos)*
3. Peça 2 — Modelagem de Dados e Requisitos a construir *(Modelagem de Sistemas 1)*
4. Peça 3 — Banco de Dados e RLS a construir *(Banco de Dados)*
5. Peça 4 — Como o Mural Conversa com a Internet a construir *(Redes de Computadores)*
6. Peça 5 — Lógica de Backend com Supabase a construir *(Desenvolvimento de Sistemas 1)*
7. Peça 6 — Construa o Aplicativo a construir *(Programação de Aplicativos)*

Visão Geral do Projeto Interdisciplinar

Cada aluno da turma constrói sua **própria cópia completa** de uma rede social da escola (o "Mural") — 1 repositório GitHub (`mural-<usuario>`) + 1 projeto Supabase próprios — ao longo de **6 matérias**, cada uma contribuindo com uma etapa real do MESMO projeto, não conteúdo isolado. Ficam de fora: Projeto de Vida, IoT e Mundo do Trabalho.

Cada etapa de cada atividade tem um **conceito** (o que aprender) seguido de um **comando** (o que fazer na prática) — mesmo padrão já usado nas trilhas TikTok (Conexão com Supabase) e ClipZone.

Escopo da 1ª versão (MVP)

- ☐ Cadastro e login
- ☐ Perfil (nome, turma, foto)
- ☐ Criar post (texto)
- ☐ Feed com posts de todos
- ☐ Curtir um post

Entidades do banco de dados

Tabela	Campos
<code>usuarios</code>	id, nome, turma, foto_url, usuario, senha, criado_em
<code>posts</code>	id, autor_id → usuarios, texto, criado_em
<code>curtidas</code>	id, post_id → posts, usuario_id → usuarios, criado_em, unique(post_id, usuario_id)

Login/cadastro seguem o mesmo padrão simplificado já usado no TikTok (tabela própria + policies de RLS, não Supabase Auth real).

Sequência pedagógica e capacidades por matéria

#	Matéria	Trilha / módulo	Capacidade (reaproveitada, nenhuma inventada)
1	Introdução de Desenvolvimento de Projetos	Nova trilha: Kickoff: Rede Social da Escola (Mural)	Reconhecer as diferentes fases pertinentes à elaboração de um projeto.
2	Modelagem de Sistemas 1	Nova trilha: Modelagem: Rede Social da Escola (Mural)	Elaborar modelo de dados e definir requisitos funcionais e não funcionais de um sistema.
3	Banco de Dados	Nova trilha: Banco de Dados: Rede Social da Escola (Mural)	Aplicar procedimentos de segurança e backup no SGBD.
4	Redes de Computadores	Módulo extra na trilha existente "Serviços de Internet e Modelos"	Reconhecer tipos e características (classificação, estrutura e modelos).

5	Desenvolvimento de Sistemas 1	Nova trilha: Projeto Mural: Lógica de Backend com Supabase	Aplicar linguagem de programação por meio de APIs, bibliotecas, frameworks na construção de rotinas de software.
6	Programação de Aplicativos	Nova trilha: Projeto: Rede Social da Escola (Mural)	Aplicar, de forma integrada, lógica de programação, manipulação de objetos e validação de formulário na construção de um aplicativo completo em JavaScript.

Como o repositório cresce

```

KICKOFF (Peça 1)
  ↓ repositório + README + issues
MODELAGEM (Peça 2)
  ↓ docs/der.png, requisitos no README
BANCO DE DADOS (Peça 3)
  ↓ database/schema.sql, database/policies.sql
REDES (Peça 4)
  ↓ (só observação, não altera o repo)
DEV. DE SISTEMAS 1 (Peça 5)
  ↓ funções JS (app.js)
PROGRAMAÇÃO DE APLICATIVOS (Peça 6)
  ↓ index.html + telas funcionando
MURAL PRONTO

```

Status desta compilação: a Peça 1 (Kickoff) já foi construída como atividade real no portal

([turmas/sistemas/atividades/projeto-mural-kickoff-trabalho.html](#) + entrada em `config.js` + teste Playwright). As Peças 2 a 6 estão especificadas abaixo, prontas para construção nas próximas fases.

Peça 1 — Kickoff do Projeto

construída

Matéria: Introdução de Desenvolvimento de Projetos

1. Apresentação

Este é o primeiro passo do projeto Mural. Aqui o aluno cuida da parte 1: criar a "casa" do projeto no GitHub e planejar o que vai ser construído nas próximas 5 matérias.

2. Situação-problema

A PixelForge Studios foi contratada pela escola pra criar uma rede social interna — o Mural —, onde alunos e professores possam se conectar, compartilhar o que estão fazendo e ver as novidades da turma. Você faz parte do time. Antes de escrever qualquer código, todo projeto de verdade passa por uma fase de organização: criar o repositório, documentar o objetivo e planejar o trabalho em tarefas.

3. Tecnologias utilizadas

GitHub (conta, repositório, README, Issues).

4. Objetivos da atividade

- Identificar em qual fase de um projeto uma atividade está.
- Criar uma conta no GitHub e um repositório.
- Escrever um README que serve de cartão de visita do projeto.
- Quebrar um projeto em tarefas (issues), uma por funcionalidade.
- Documentar decisões de projeto desde o início.

Etapa 1 — Missão do projeto

CONCEITO

O Mural é um projeto real que você vai construir aos poucos, matéria por matéria, ao longo do curso.

COMANDO

Leia as 5 funcionalidades da 1ª versão e escreva com suas palavras o que o Mural vai fazer.

Etapa 2 — Fases de um projeto, agora na prática

CONCEITO

Você já estudou, na Oficina de Comunicação, as fases de elaboração de um projeto. O Kickoff do Mural é a fase de planejamento virando ação de verdade.

COMANDO

Identifique em qual fase o projeto está agora, e em qual fase ele vai estar quando a Modelagem de Sistemas terminar o DER.

Etapa 3 — Controle de versão e GitHub

CONCEITO

Times de desenvolvimento usam controle de versão (Git) pra guardar o histórico de tudo que foi feito. O GitHub hospeda esse histórico e funciona como portfólio.

COMANDO

Crie sua conta no GitHub (se ainda não tiver uma).

Etapa 4 — Criar o repositório do Mural

CONCEITO

Um repositório é a "casa" do projeto: todo código, documentação e histórico moram ali.

COMANDO

Crie um repositório novo, público, chamado `mural-<seuusuario>`, já marcando a opção de criar um `README.md`.

Etapa 5 — README como cartão de visita

CONCEITO

O README é a primeira coisa que qualquer pessoa vê ao abrir o repositório.

COMANDO

Escreva no README: objetivo do Mural, as 5 funcionalidades da 1ª versão, e as tecnologias (HTML, CSS, JavaScript, GitHub, Supabase).

Etapa 6 — Issues como plano de trabalho

CONCEITO

Issues são tarefas — o jeito de quebrar um projeto grande em pedaços pequenos e acompanháveis.

COMANDO

Crie 5 issues, uma por funcionalidade: "Cadastro e login", "Perfil do usuário", "Criar post", "Feed com posts de todos", "Curtir post".

Etapa 7 — Documentar decisões desde o início

CONCEITO

Decisões de projeto documentadas desde o começo evitam retrabalho.

COMANDO

Adicione ao README uma seção "Decisões de Projeto" com o nome escolhido pro app e o público-alvo.

Checklist de entrega

- ☐ Conta no GitHub criada.
- ☐ Repositório `mural-<usuario>` criado (público, com README).
- ☐ README com objetivo, as 5 funcionalidades, tecnologias e "Decisões de Projeto".
- ☐ 5 issues criadas (uma por funcionalidade).

Capacidade principal

Reconhecer as diferentes fases pertinentes à elaboração de um projeto.

Observação para o docente: reaproveita a mesma capacidade já trabalhada pela trilha "Oficina de Comunicação" (mesma matéria), com produto diferente: aqui o produto é um repositório GitHub real, reaproveitado pelas próximas 5 matérias do projeto Mural.

Peça 2 — Modelagem de Dados e Requisitos

a construir

Matéria: Modelagem de Sistemas 1

1. Apresentação

2ª etapa do projeto Mural. Aqui o aluno modela, em cima do repositório `mural-<usuario>` já criado, os requisitos e os dados que vão virar as tabelas reais na próxima matéria (Banco de Dados).

2. Situação-problema

A PixelForge Studios já tem o repositório do Mural criado (Kickoff). Antes de criar qualquer tabela ou tela, o time de modelagem precisa definir o que o sistema precisa fazer (requisitos) e como os dados vão se organizar (entidades, atributos, relacionamentos).

3. Tecnologias utilizadas

Canva / Google Slides / draw.io / dbdiagram.io (à escolha, pra desenhar o DER); GitHub (repositório já existente).

4. Objetivos da atividade

- Diferenciar requisito funcional de não funcional.
- Identificar entidade, atributo e relacionamento.
- Definir cardinalidade (1:N, N:N).
- Marcar chave primária e chave estrangeira.
- Desenhar um DER/MER.

Etapa 1 — Requisitos funcionais e não funcionais

CONCEITO

Requisito funcional é uma AÇÃO que o sistema faz; requisito não funcional é uma QUALIDADE do sistema.

COMANDO

Liste 5 requisitos funcionais (um por funcionalidade do MVP) e 3 requisitos não funcionais do Mural.

Etapa 2 — Entidades e atributos

CONCEITO

Entidade é algo do mundo real que vira tabela; atributo é uma característica dela, que vira coluna.

COMANDO

Liste os atributos de Usuário, Post e Curtida.

Etapa 3 — Relacionamentos e cardinalidade

CONCEITO

Cardinalidade descreve quantos registros de uma entidade se relacionam com quantos de outra.

COMANDO

Defina a cardinalidade Usuário-Post e Post/Usuário-Curtida.

Etapa 4 — Chave primária e chave estrangeira

CONCEITO

A chave primária identifica um registro de forma única; a chave estrangeira cria o link entre tabelas.

COMANDO

Marque a PK de cada entidade e as FKs entre Post→Usuário e Curtida→Post/Usuário.

Etapa 5 — Desenhar o DER

CONCEITO

O DER/MER é o "mapa" visual do banco de dados antes de criar qualquer tabela.

COMANDO

Desenhe o DER, exporte como imagem e salve como `docs/der.png` no repositório (commit + push).

Etapa 6 — Entrega e questionário

COMANDO

Exporte o material, envie por e-mail pro professor e peça o visto. Depois responda o questionário sobre o que foi modelado (mesmo padrão trabalho→questionário já existente, com trava anti-saída de tela).

Checklist de entrega

- ☐ 5 requisitos funcionais + 3 não funcionais listados.
- ☐ Entidades Usuário/Post/Curtida com atributos definidos.
- ☐ Cardinalidade definida entre as 3 entidades.
- ☐ PK/FK marcadas.
- ☐ DER desenhado, exportado e commitado em `docs/der.png`.
- ☐ Questionário respondido (≥80%).

Capacidade principal

Elaborar modelo de dados e definir requisitos funcionais e não funcionais de um sistema.

Observação para o docente: capacidade provisória (TODO), mesma já usada na trilha "Modelagem de Dados e Requisitos" — confirmar o texto real da MSEP antes de liberar pra turma.

Peça 3 — Banco de Dados e RLS

a construir

Matéria: Banco de Dados

1. Apresentação

3ª etapa do projeto Mural. O aluno cria, num projeto Supabase próprio, as 3 tabelas definidas no DER da Modelagem, com chaves estrangeiras e políticas de RLS — mesmo padrão já usado na trilha "Conexão com Supabase" (TikTak), agora aplicado ao Mural.

2. Situação-problema

O DER do Mural já existe. Agora é preciso transformar esse modelo em um banco de dados de verdade, no Supabase, seguro (com RLS) — pronto pra receber as funções de backend da próxima matéria.

3. Tecnologias utilizadas

Supabase (projeto, SQL Editor, Data API, RLS/Policies); GitHub (commit dos scripts SQL).

4. Objetivos da atividade

- Criar um projeto Supabase.
- Criar tabelas com FK entre elas.
- Testar INSERT/SELECT manualmente.
- Habilitar RLS e criar policies de leitura/inserção/atualização restrita ao autor.
- Commitar os scripts SQL no repositório.

Etapa 1 — Criar o projeto Supabase

COMANDO

Crie um novo projeto Supabase chamado `mural`.

Etapa 2 — Tabela usuarios

CONCEITO

Toda tabela precisa de uma chave primária única.

COMANDO

```
CREATE TABLE usuarios com id (PK, auto), nome, turma, foto_url, usuario, senha, criado_em. Committe em database/schema.sql .
```

Etapa 3 — Tabela posts (com FK)

CONCEITO

Chave estrangeira (FK) conecta uma linha de uma tabela a uma linha de outra.

COMANDO

```
CREATE TABLE posts com id (PK), autor_id (FK → usuarios), texto, criado_em.
```

Etapa 4 — Tabela curtidas (com restrição única)

CONCEITO

Um unique constraint impede duplicidade — evita que o mesmo usuário curta o mesmo post duas vezes.

COMANDO

```
CREATE TABLE curtidas com id (PK), post_id (FK), usuario_id (FK), criado_em, unique(post_id, usuario_id) .
```

Etapa 5 — Testar manualmente

COMANDO

Insira 1 usuário, 1 post e 1 curtida pelo SQL Editor ou pela interface de tabela. Confira que os relacionamentos foram respeitados.

Etapa 6 — Revisão da Data API

CONCEITO

A Data API transforma comandos SQL em chamadas que o JavaScript do navegador consegue fazer (revisão do TikTok).

COMANDO

Localize a Project URL e a Publishable Key do projeto `mural` .

Etapa 7 — Habilitar RLS

CONCEITO

Sem RLS habilitado, o banco recusa qualquer acesso pela Data API, mesmo com a chave certa.

COMANDO

Habilite RLS nas 3 tabelas.

Etapa 8 — Policies de leitura e inserção

COMANDO

Crie policies de leitura pública e de inserção liberada nas 3 tabelas. Committe em `database/policies.sql`.

Etapa 9 — Policy de atualização/exclusão restrita ao autor

COMANDO

Crie uma policy de update/delete em posts que só permita a operação quando autor_id bate com o usuário que faz a requisição.

Etapa 10 — Testar as policies e visto

COMANDO

Teste select/insert/update de cada tabela. Chame o professor pra dar o visto.

Capacidade principal

Aplicar procedimentos de segurança e backup no SGBD.

Peça 4 — Como o Mural Conversa com a Internet

a construir

Matéria: Redes de Computadores (módulo extra em "Serviços de Internet e Modelos", não trilha nova)

1. Apresentação

4ª etapa do projeto Mural. O aluno usa o próprio Mural (banco já criado) pra observar, na prática, como o navegador conversa com a internet.

2. Situação-problema

O time de Redes precisa garantir que a comunicação entre o navegador e o servidor do Mural (Supabase) é entendida e monitorável.

3. Objetivos da atividade

- Identificar método, URL, headers e corpo de uma requisição HTTP real.
- Explicar o papel do DNS.
- Reconhecer HTTPS.
- Ler status codes e relacioná-los a RLS.
- Relacionar a requisição à camada de aplicação do modelo TCP/IP.

Etapa 1 — Requisição HTTP

CONCEITO

Uma requisição HTTP tem método, URL, headers e corpo.

COMANDO

Abra o DevTools (Network) com o Mural rodando e encontre a requisição pro Supabase. Anote método, URL e 2 headers.

Etapa 2 — DNS

CONCEITO

DNS traduz um domínio em endereço IP.

COMANDO

Identifique o domínio `*.supabase.co` e explique o que quebraria sem DNS.

Etapa 3 — HTTPS

CONCEITO

HTTPS criptografa a comunicação — essencial quando trafegam senhas.

COMANDO

Confira o cadeado e o protocolo (https) na requisição.

Etapa 4 — Status code

CONCEITO

200, 201, 401, 403 dizem o resultado da requisição.

COMANDO

Force um erro (ex.: desative uma policy de RLS) e leia o status code retornado.

Etapa 5 — Camada de aplicação

CONCEITO

No modelo TCP/IP, HTTP roda na camada de aplicação.

COMANDO

Relacione a requisição observada com essa camada.

Capacidade principal

Reconhecer tipos e características (classificação, estrutura e modelos).

Peça 5 — Lógica de Backend com Supabase

a construir

Matéria: Desenvolvimento de Sistemas 1

1. Apresentação

5ª etapa do projeto Mural. Com as tabelas já criadas, o aluno escreve, no mesmo repositório (Codespace), as funções JavaScript com `supabase-js` que implementam as 5 funcionalidades do MVP.

2. Objetivos da atividade

- Importar e usar uma biblioteca/SDK de terceiros.
- Escrever funções assíncronas (async/await).
- Fazer INSERT, SELECT com filtro, SELECT com join e ordenação.
- Tratar erros retornados pela API.
- Evitar duplicidade em uma operação (curtir 1x).

Etapa 1 — Biblioteca vs. requisição na mão

COMANDO

Adicione `supabase-js` ao projeto e crie o cliente com URL e Publishable Key do projeto `mural`.

Etapa 2 — cadastrarUsuario

COMANDO

Escreva `async function cadastrarUsuario(nome, turma, usuario, senha)` que faz insert em usuarios.

Etapa 3 — fazerLogin

COMANDO

Escreva `async function fazerLogin(usuario, senha)` que filtra por usuário e senha.

Etapa 4 — criarPost

COMANDO

Escreva `async function criarPost(autorId, texto)`.

Etapa 5 — carregarFeed

COMANDO

Escreva `async function carregarFeed()` que retorna os posts com o nome do autor, mais recentes primeiro (join + ordenação).

Etapa 6 — curtir

COMANDO

Escreva `async function curtir(postId, usuarioId)` tratando a duplicidade.

Etapa 7 — Tratamento de erro

COMANDO

Adicione try/catch e verificação de `error` em todas as funções acima.

Etapa 8 — Teste manual e commit

COMANDO

Teste cada função pelo console, capture evidência e commite `app.js / mural.js`. Peça o visto.

Capacidade principal

Aplicar linguagem de programação por meio de APIs, bibliotecas, frameworks na construção de rotinas de software.

Peça 6 — Construa o Aplicativo

a construir

Matéria: Programação de Aplicativos

1. Apresentação

6ª e última etapa. O aluno constrói a interface (HTML/CSS/JS) que chama as funções REAIS escritas na matéria anterior, entregando o app funcionando de ponta a ponta.

2. Objetivos da atividade

- Construir formulários com validação.
- Renderizar uma lista dinâmica (feed) a partir de dados reais.
- Ligar eventos de clique/submit às funções de backend já existentes.
- Implementar um toggle de estado (curtir/descurtir) com contador.
- Entregar um aplicativo completo, integrado, funcionando com dados reais.

Etapa 1 — Tela de Cadastro

COMANDO

Construa o formulário e `validarCadastro()` ; ao validar, chame `cadastrarUsuario()` .

Etapa 2 — Tela de Login

COMANDO

Construa a tela e `validarLogin()` , chamando `fazerLogin()` .

Etapa 3 — Tela de Perfil

COMANDO

Exiba nome, turma e foto do usuário logado.

Etapa 4 — Feed

COMANDO

Implemente `renderPost()` pra cada item de `carregarFeed()` , e `formatarData()` .

Etapa 5 — Criar Post

COMANDO

Construa o formulário de texto + botão publicar, chamando `criarPost()` e atualizando o feed.

Etapa 6 — Curtir

COMANDO

Implemente o botão de curtir, chamando `curtir()` e atualizando o contador.

Etapa 7 — Tema visual

COMANDO

Escolha uma paleta de cores/tema pro Mural (como no ClipZone).

Etapa 8 — Visto do professor

COMANDO

Chame o professor pra conferir o app funcionando de ponta a ponta: cadastro → login → postar → curtir.

Capacidade principal

Aplicar, de forma integrada, lógica de programação, manipulação de objetos e validação de formulário na construção de um aplicativo completo em JavaScript.